

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE CODE: CSA02**

**COURSE NAME: C Programming**

---

### COURSE OUTCOME COVERED

**CO1:** Construct structured C programs using constants, variables, data types, and preprocessor directives to address basic computational tasks. (BL2, BL3)

*Assessment Tool Weightage: 40%*

---

**QUESTIONS:**

**Total Marks:100**

Annexure A

### PSEUDO CODE WRITING

---

#### ***Code of Conduct:***

*I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## Annexure A

### PSEUDO CODE WRITING

Q1. Given pseudocode:

```
#define DEBUG 0
```

```
IF DEBUG
```

```
    PRINT intermediate_value
```

```
END IF
```

- Identify error in directive usage
- Correct the logic

Q2. #define PI \_\_\_\_\_

```
DECLARE radius AS float
```

```
DECLARE area AS float
```

```
INPUT radius
```

```
area = _____
```

```
PRINT area
```

Fill:

- Constant
- Formula

Q3. #define RATE 1.5

```
DECLARE x AS int = 10
```

```
DECLARE result AS float
```

```
result = x * RATE
```

```
PRINT result
```

Questions:

- Output?
- Why is the result float?

Q4. #define MAX 1000

DECLARE temp AS int  
INPUT temp

IF temp > MAX  
    PRINT "Valid"  
ELSE  
    PRINT "Invalid"

- Identify logical error
- Suggest correct condition

Q5. Given unordered pseudocode steps:

- DEFINE conversion rate
- PRINT result
- INPUT amount
- CALCULATE converted value

Arrange in correct order

Q6. #define MIN 10  
    #define MAX 50

DECLARE value AS int  
INPUT value

\_\_\_\_\_ (value >= MIN AND value <= MAX)  
PRINT "Within range"

\_\_\_\_\_  
PRINT "Out of range"

Fill missing control structures

Q7. A billing system includes optional features such as discount calculation and tax computation. Depending on the business requirement, these features may be enabled or disabled at compile time. The program should calculate the total bill amount based on input price and quantity. If discount and tax features are enabled, they should be applied using predefined constants; otherwise, they should be skipped. The system must ensure that floating-point precision is maintained during calculations.

Write pseudocode using preprocessor directives to enable/disable features, define constants for discount and tax rates, and use appropriate data types.

Q8. #define TAX 5%

DECLARE price AS int  
DECLARE total AS int

total = price + (price \* TAX)

PRINT total

- Identify errors
- Correct data types and constant definition

9. A temperature conversion program supports multiple modes such as Celsius to Fahrenheit and Fahrenheit to Celsius. The mode of operation should be selected at compile time using preprocessor directives. The program should use appropriate floating-point data types to ensure accuracy and define conversion formulas using symbolic constants wherever applicable. The final output should be displayed with two decimal precision.

Write pseudocode using preprocessor directives for mode selection, constants for conversion formulas, and proper data types.

10. #define SIZE 5

DECLARE arr[SIZE]  
INPUT elements

- Extend pseudocode to:
  - Find sum
  - Compute average

## RUBRICS FOR CALCULATION

| Criteria                  | Weightage | Level 4 – Exemplary                                                   | Level 3 – Proficient                      | Level 2 – Developing         | Level 1 – Beginning             |
|---------------------------|-----------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20%       | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30%       | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20%       | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10%       | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20%       | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |